

PROJECT NAME:

Booogle (E-tickting)

EXPERIMENT 1

Objective 1: Problem Analysis and Project Planning

- **Problem Statement:**

The Ticketing Division of the transportation sector requires a system to streamline its ticket booking and management processes while providing a user-friendly interface for its customers. The existing manual or semi-automated systems fail to meet customer expectations for real-time ticket availability, booking, and payment processing. This leads to inefficiencies in service delivery, delays, and reduced customer satisfaction. A robust e-ticketing system is essential to overcome these challenges and enhance the overall user experience.

- **Project Scope:**

The system will automate key ticketing functions, including ticket booking, cancellation, rescheduling, and payment processing. The project will enhance the user experience through a secure, user-friendly interface that integrates with a database for real-time ticket availability and transaction handling.

- **Infrastructure Required:**

A web-based e-ticketing system with a secure and robust database backend, SSL encryption to ensure data security, and 24/7 availability. The system should be capable of handling high traffic during peak booking times and allow for future scalability to accommodate growing user demand.

EXPERIMENT 2

Objective 2: Software Requirement Specification (SRS):

- **Purpose:-**

The purpose of this SRS document is to define the functional and non-functional requirements of the E-Ticketing System. The system is designed to automate ticket booking and management processes, providing both customers and administrators with a secure, user-friendly platform for reserving tickets, managing bookings, and handling various other ticketing operations.

- **Scope:-**

This project focuses on developing a secure web-based application that facilitates real-time ticketing operations. It is aimed at automating key activities such as ticket booking,

cancellations, rescheduling, seat selection, and payment processing. The system will also include administrative functionalities, such as managing user accounts, generating reports, monitoring transaction history, and ensuring secure data handling. Additionally, the system will support notifications, integration with payment gateways, and scalability to meet increasing user demand.

Modules/Phases:

1. Ticket Booking Module:

Customers log in using their credentials to perform operations such as searching for available tickets, booking tickets, selecting seats, viewing booking history, rescheduling, and canceling tickets.

2. E-Ticketing Administrator Module:

Administrators log in to manage customer accounts, monitor ticket availability, update travel schedules, approve refund requests, handle cancellations, and generate reports on ticket sales and system performance.

Functional Requirements:

- Ticket Availability Retrieval
- Ticket Booking and Payment
- Ticket Cancellation and Refunds
- Travel Schedule Viewing
- Booking History and Notifications

Non-Functional Requirements:

- Secure Access:
- 24/7 Availability:
- High Performance:
- Scalable Architecture

2. Overall Description

Product Perspective :-

The E-Ticketing System is designed to automate ticket booking and management processes, enhance customer convenience, and ensure secure transactions. It is a web-based application that supports both customers and administrators, enabling seamless ticketing operations and efficient system management.

Product Features :-

- **User-Friendly Interface:** Provides an intuitive interface for customers to search for tickets, book or cancel them, and manage bookings.
- **Real-Time Updates:** Ensures real-time information on ticket availability, travel schedules, and booking statuses.

- **Secure Transactions:** Incorporates secure data handling with multi-level authentication and SSL encryption.
- **Administrative Tools:** Includes functionalities for managing travel schedules, monitoring ticket availability, handling customer requests, and generating detailed reports.

User Classes and Characteristics :-

- **Customers:** Individuals who can log in to search for tickets, make bookings, reschedule or cancel tickets, and view their booking history.
- **Administrators:** System personnel responsible for managing user accounts, updating travel schedules, monitoring ticket sales, processing refund requests, and ensuring system efficiency.

Operating Environment

- **Platform:** Web-based application.
- **Operating System:** Compatible with Windows, Linux, and macOS platforms using a web browser.
- **Browser Compatibility:** Supports Chrome, Firefox, Edge, and Safari.
- **Database:** Utilizes an SQL-based database management system (e.g., MySQL, PostgreSQL, or Oracle) for efficient data handling.
- **Security:** Ensures SSL encryption for all user sessions to protect sensitive data.
- **Design and Implementation Constraints**
- **Multi-User Access:** The system must support simultaneous access by multiple users without performance degradation.

Design and Implementation Constraints for E-Ticketing System

- **Data Encryption:** All sensitive data (e.g., payment information, personal user data) must be encrypted during transmission (e.g., via TLS) and at rest.
- **Authentication Mechanisms:** Multi-level authentication methods such as OTPs (One-Time Passwords), 2FA (Two-Factor Authentication), or biometric authentication must be integrated to ensure secure user access.
- **Access Control:** Role-based access control (RBAC) should be implemented to restrict system functionality based on user roles.
- **Compliance Standards:** The system must comply with relevant regulations such as PCI DSS for payment data security and GDPR for data privacy.

EXPERIMENT 3

Objective 3: Data Modeling

Data Dictionary

1. Data Dictionary

Customer table

Name	Null?	Type
Customer_id(PK)	Not Null	Integer
Cust_first_name		VARCAR(20)
Cust_last_name		VARCAR(20)
DOB		VARCAR(20)
Gender		VARCAR(20)

Login table

Name	Null?	Type
Customer_id(FK)		INTEGER
Password		VARCAR(20)
Username		VARCAR(20)

Customer Detail table

Name	Null?	Type
Customer_id(FK)	Not Null	Integer
City		VARCAR(20)
State		VARCAR(20)
Phone number		INTEGER
Email_id		

Account table

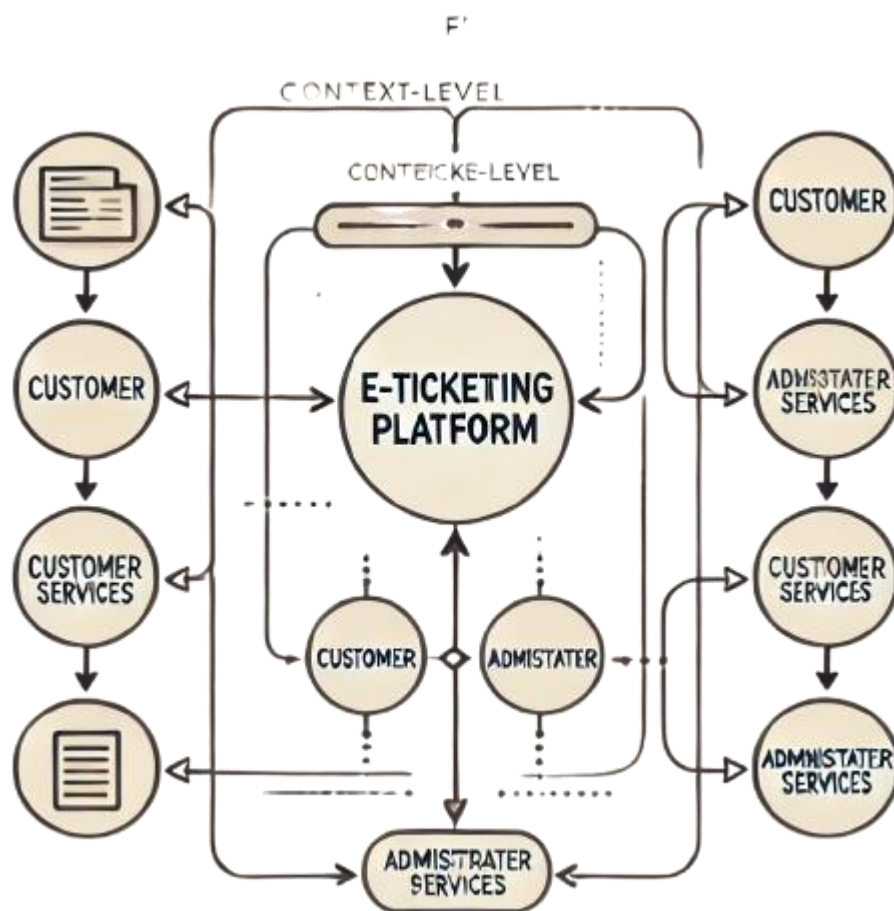
Name	Null?	Type
Account Number(PK)	NOT NULL	NUMBER(8)
Customer_id(FK)	NOT NULL	INTEGER

Type of account		VARCAR(20)
-----------------	--	------------

Transaction type table

Name	Null?	Type
Transaction Number(PK)	NOT NULL	INTEGER
Account Number(FK)	NOT NULL	INTEGER

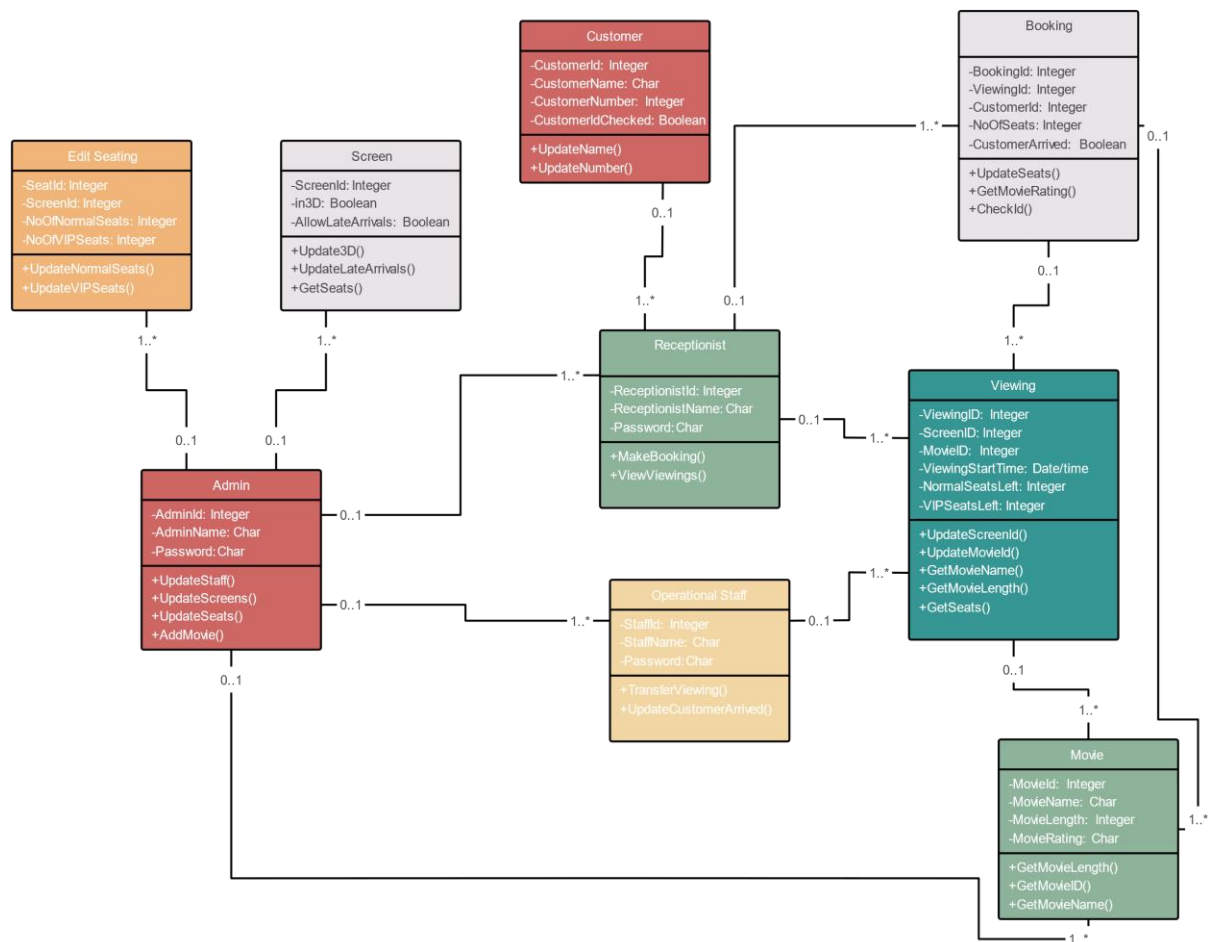
2. Context Level Diagram



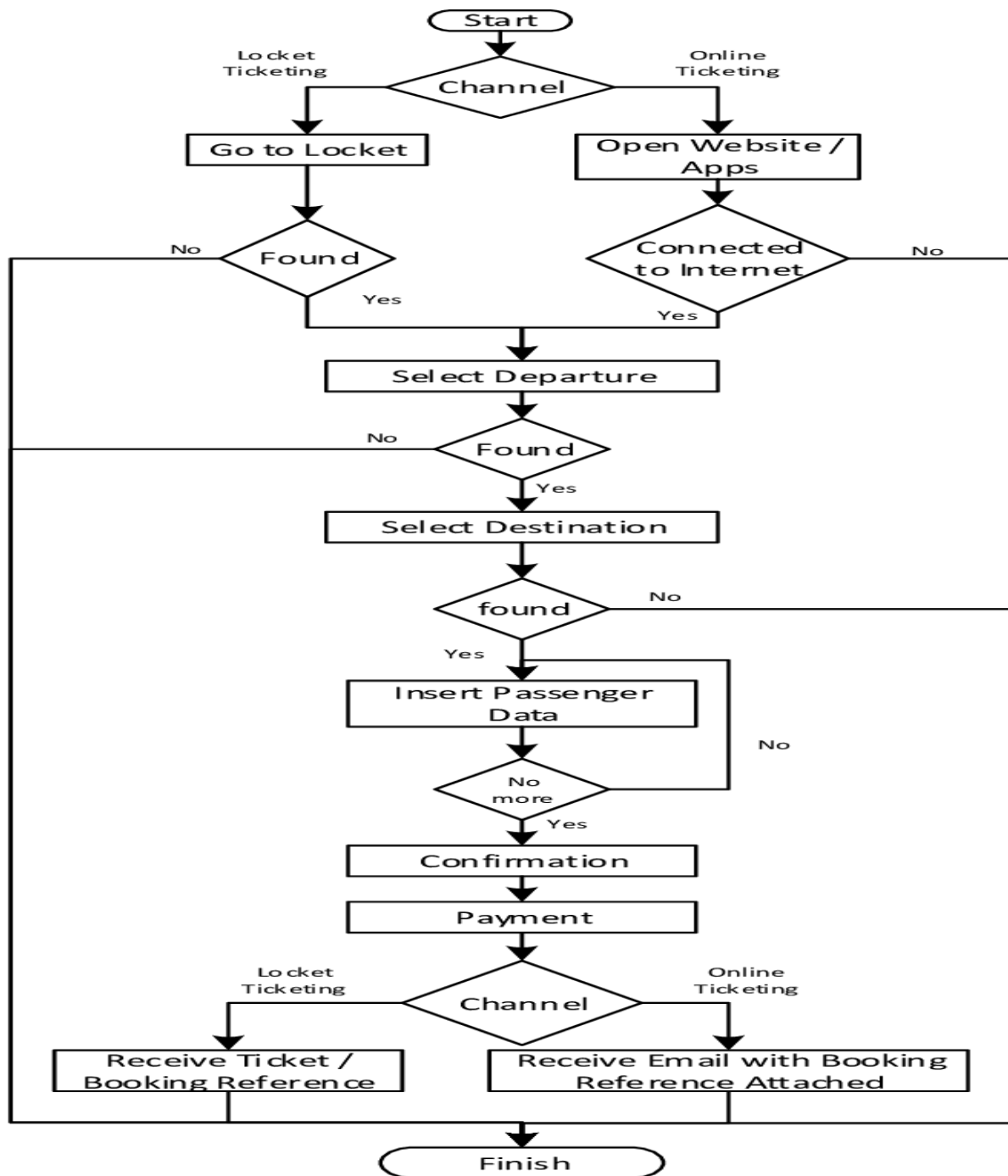
EXPERIMENT 4

Objective 4: Software Designing

1. Class diagram

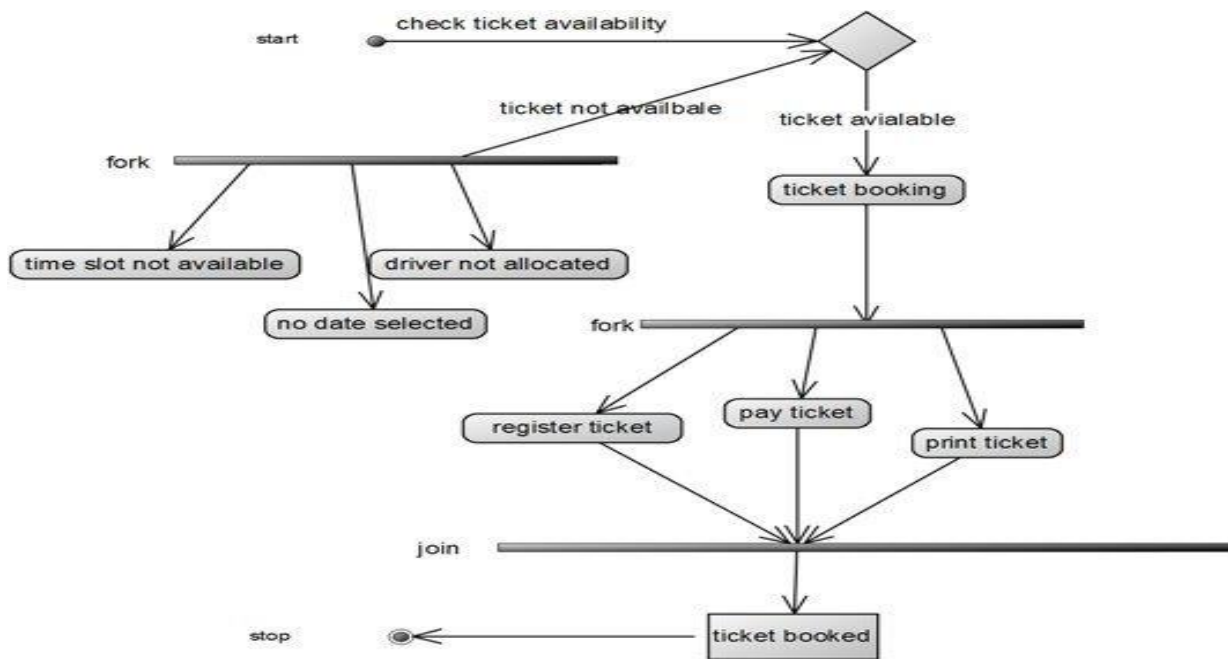


2. Use Case Diagram

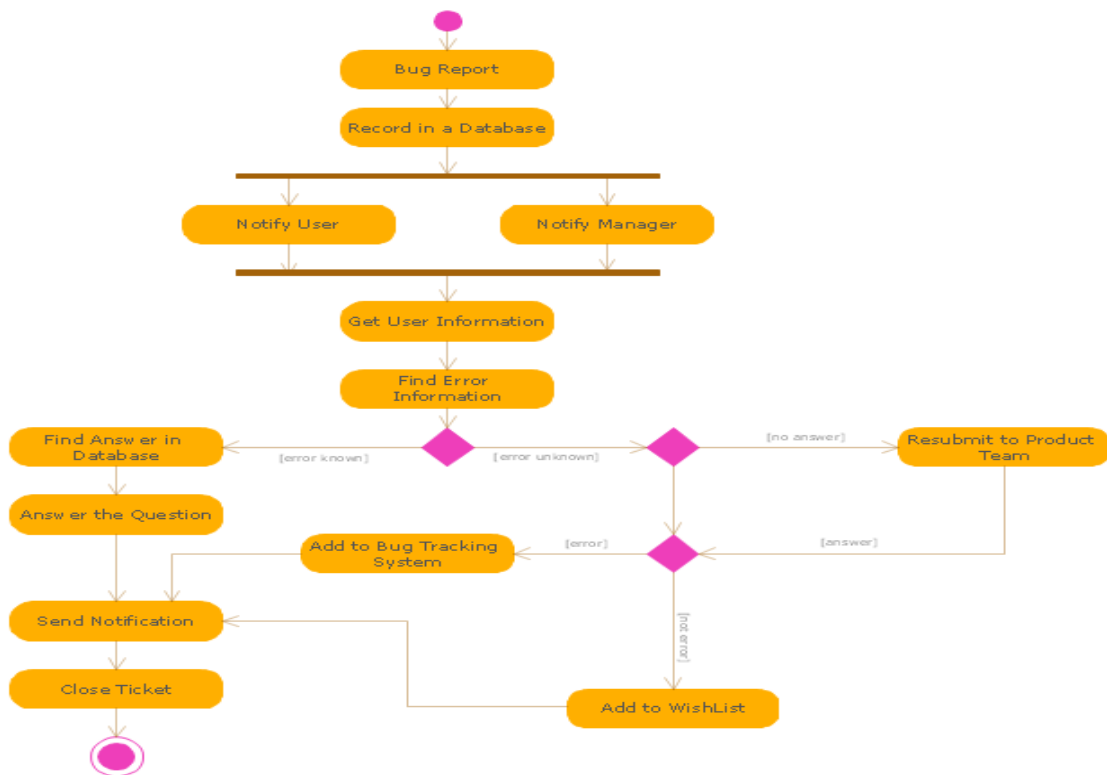


3)Activity Diagram

(3.1) Customer Activity Diagram



(3.2) Activity Diagram for Administrator



EXPERIMENT 6

Objective: Software Requirement Analysis – Describe the individual phases/modules of the project and identify deliverables. Identify functional and non-functional requirements.

Software Requirement Analysis

Modules/Phases:

1. User Ticketing Module

- Description: This module enables users to log in and manage their ticket bookings. Users can search for events, book tickets, view booking history, and update their personal information.

- Key Features:

- User login: Users authenticate using their credentials.

- Event search and booking: Users can browse and book tickets for various events or shows.
- Booking history: Users can view details of their past ticket bookings.
- Personal information management: Users can update their personal details.
- E-ticket generation: Users receive e-tickets via email or download after successful booking.

2. Event Organizer Module

- Description: This module allows event organizers to manage event listings and monitor ticket sales.
- Key Features:
 - Organizer login: Event organizers authenticate using their credentials.
 - Event management: Organizers can create, update, and delete event details.
 - Ticket availability management: Organizers can manage ticket inventory and pricing.
 - Sales report generation: Organizers can generate reports on ticket sales and revenue.

Functional Requirements:

1. Event search

- The system shall allow users to search for events based on location, date, and type.

2. Ticket booking

- The system shall enable users to select their preferred seats (if applicable) and book tickets securely.

3. E-ticket delivery

- The system shall generate and deliver e-tickets to users via email or provide an option to download them.

4. Cancellation and refunds

- The system shall allow users to cancel bookings and process refunds as per event policies.

5. Booking history

- The system shall allow users to view their previous bookings, including event details and transaction information.

Non-Functional Requirements:

1. Performance: The system should support simultaneous bookings by multiple users without any downtime.

2. Scalability: The platform should scale to accommodate a growing number of events and users.

3. Security: Ensure secure payment processing and user data protection through encryption.

4. Usability: Provide a user-friendly interface for seamless navigation.

5. Availability: The platform should maintain 99.9% uptime to ensure continuous availability.

EXPERIMENT- 7

Objective:

Testing phase-related documents.

1. Design Documents

- Purpose: Outline the architecture and design of the software solution.
- Components:
 - o Architecture Design Document: Details the overall system architecture, including high-level components and their interactions.
 - o Detailed Design Document: Provides in-depth descriptions of modules, interfaces, and data structures.
 - o User Interface Design Document: Outlines the UI/UX aspects, including wireframes and design elements.
 - o Database Design Document: Contains the schema design, entity-relationship diagrams, and data dictionary.
 - o Component Design Specifications: Individual design specifications for each module, including algorithms, methods, and performance considerations.

2. Testing Phase-Related Documents

- Purpose: Ensure the software meets specified requirements and is free of defects.
- Components:
 - o Test Plan: Describes the scope, approach, resources, and schedule for testing activities.
 - o Test Case Document: Lists test cases to be executed, including input data, expected results, and execution conditions.
 - o Test Script: Detailed scripts for automated testing, specifying the steps and expected outcomes.

- o Defect Tracking Document: Records defects identified during testing, including status, severity, and resolution steps.

- o Test Summary Report: Provides an overview of testing activities, outcomes, and recommendations for release.

3. Integration of Design and Testing Documents

- Ensure that the design documents support the testing phase by clearly defining requirements and specifications that testing must validate.

- Use traceability matrices to link requirements in design documents to corresponding test cases.

4. Tools and Techniques

- Consider using tools such as UML for design documentation and test management software (e.g., JIRA, TestRail) for managing testing documents.

- Implement version control systems (e.g., Git) for managing changes to design documents.

EXPERIMENT 8

Objective: Preparation of Software Configuration Management and Risk Management-related documents.

1. Software Configuration Management (SCM) Documents

- Purpose: Ensure the integrity, consistency, and traceability of software throughout its lifecycle.
- Components:
 - o Configuration Management Plan:
 - Overview: Outlines the processes and practices for managing software configurations.
 - Key Sections:
 - Configuration identification
 - Version control procedures
 - Change management processes
 - Configuration audits and reviews
 - o Configuration Item (CI) List:
 - Defines all configuration items, including software components, documentation, and hardware.
 - Specifies version numbers and status of each item.
 - o Change Request Document:
 - Captures requests for changes to software configurations, including justification and impact analysis.
 - o Baseline Documents:
 - Establishes formal baselines for software releases, including functional and non-functional requirements.
 - Describes the process for establishing and approving baselines.
 - o Configuration Status Report:
 - Provides an overview of the current status of all configuration items and any outstanding change requests.

2. Risk Management Documents

- Purpose: Identify, analyze, and mitigate risks throughout the software development lifecycle.
- Components:
 - o Risk Management Plan:
 - Overview: Outlines the approach for managing risks, including methodologies and roles.
 - Key Sections:
 - Risk identification processes
 - Risk assessment techniques
 - Risk mitigation strategies
 - Monitoring and reporting procedures
 - o Risk Register:
 - Maintains a log of identified risks, including descriptions, impact assessments, likelihood, and mitigation strategies.
 - Includes risk owners and status updates.
 - o Risk Assessment Matrix:
 - Visual representation of risks, categorizing them based on likelihood and impact.
 - Helps prioritize risks for further action.
 - o Risk Mitigation Plan:
 - Details specific strategies for addressing high-priority risks.
 - Includes contingency plans and resources required for implementation.
 - o Risk Monitoring Report:
 - Provides updates on risk statuses, including newly identified risks and changes to existing risks.
 - Summarizes the effectiveness of mitigation strategies.

3. Integration of SCM and Risk Management

- Establish a link between configuration management and risk management by identifying configuration items that are critical to mitigating specific risks.
- Ensure that changes in software configurations are assessed for their impact on identified risks.

4. Tools and Techniques

- Consider using version control systems (e.g., Git, Subversion) for SCM and risk management software (e.g., RiskWatch, RiskyProject) for tracking and managing risks.
- Utilize project management tools (e.g., JIRA, Trello) to integrate SCM and risk management activities into the overall project workflow.

EXPERIMENT 9

Objective: Study and use a Design phase CASE tool to develop an e-ticketing platform.

Overview of CASE Tools for E-Ticketing Platform

- Definition: Computer-Aided Software Engineering (CASE) tools assist in software development by providing automated support for software design, coding, testing, and maintenance of an e-ticketing platform.
- Types of CASE Tools:
 - Upper CASE Tools: Focus on the early stages of e-ticketing platform development, such as requirements gathering and design.
 - Lower CASE Tools: Support coding, testing, and maintenance of the e-ticketing platform.
 - Integrated CASE Tools: Provide support throughout the entire e-ticketing platform development lifecycle.

Key Features of Design Phase CASE Tools for E-Ticketing Platform

- Diagramming Capabilities: Support for UML diagrams (e.g., use case diagrams, class diagrams, sequence diagrams) to model the e-ticketing platform's functionality.
- Code Generation: Ability to generate code from design specifications for the e-ticketing platform.
- Documentation Generation: Automated generation of design documentation for the e-ticketing platform.
- Version Control: Management of design changes and revisions for the e-ticketing platform.

- Collaboration Features: Support for teamwork and communication among developers working on the e-ticketing platform.

Popular Design Phase CASE Tools for E-Ticketing Platform

- StarUML: A UML modeling tool that supports various diagram types and allows for code generation for the e-ticketing platform.
- Visual Paradigm: A comprehensive tool that supports modeling, code generation, and project management for the e-ticketing platform.
- Enterprise Architect: Offers extensive support for UML and other modeling standards, along with project management features for the e-ticketing platform.
- Lucidchart: A web-based diagramming tool that facilitates collaborative design and sharing for the e-ticketing platform.

Performing the Design Using a CASE Tool for E-Ticketing Platform

1. *Selecting a CASE Tool*: Choose a CASE tool based on your e-ticketing platform project requirements, team preferences, and budget.
2. *Design Process Using a CASE Tool*:
 - Step 1: Define Requirements
 - Gather and document functional and non-functional requirements for the e-ticketing platform.
 - Step 2: Create Use Case Diagrams
 - Use the CASE tool to create use case diagrams that capture user interactions with the e-ticketing platform.
 - Step 3: Develop Class Diagrams

- Design the class structure by creating class diagrams that represent the e-ticketing platform's entities and relationships.

- Step 4: Sequence Diagrams

- Model the interactions between objects over time using sequence diagrams, illustrating how the e-ticketing platform behaves in various scenarios.

- Step 5: Generate Documentation

- Utilize the tool's documentation features to generate design documents that outline the e-ticketing platform's architecture, diagrams, and specifications.

- Step 6: Review and Iterate

- Conduct design reviews with stakeholders to gather feedback and make necessary adjustments.

- Step 7: Code Generation (if applicable)

- Use the code generation feature of the CASE tool to produce initial code from the design models for the e-ticketing platform.

3. *Collaboration and Version Control*:

- Encourage team collaboration through shared workspaces in the CASE tool.

- Utilize version control features to manage changes and track the evolution of the e-ticketing platform's design.

EXPERIMENT 10

Objective: Develop test cases for unit testing and integration testing
Unit Testing Test Cases.

Unit Testing Test Cases

Test Case ID	Test Case Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_E1	Test Login Function	Verify that the user can successfully log in.	User account is registered and active	1. Call login ("user@example.com", "password123"). 2. Observe if the user is authenticated.	Login is successful; user directed to dashboard.	(To be filled)	(To be filled)
TC_E2	Test Ticket Search	Verify that the search function works for tickets.	Database contains tickets for a given route/date.	1. Call search Tickets ("Delhi", "Mumbai", "2024-12-01"). 2. Observe the search results.	Relevant tickets are displayed.	(To be filled)	(To be filled)
TC_E3	Test Add to Cart Function	Verify that a ticket can be added to the cart.	User is logged in, and search results are displayed.	1. Call addToCart(ticketID). 2. Observe the cart status.	Ticket is successfully added to the cart.	(To be filled)	(To be filled)

Integration Testing Test Cases

Test Case ID	Test Case Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_I1	Test Add Cash and Check Balance Integration	Verify that adding cash updates the cash balance correctly.	The cash counter is initialized with a balance of 0.00.	1. Call addCash(100.00). 2. Call checkBalance(). 3. Observe the returned balance.	Returned balance = 100.00	(To be filled)	(To be filled)
TC_I2	Test Remove Cash and Check Balance Integration	Verify that removing cash updates the cash balance correctly.	The cash counter has a balance of 200.00.	1. Call removeCash(50.00). 2. Call checkBalance(). 3. Observe the returned balance.	Returned balance = 150.00	(To be filled)	(To be filled)
TC_I3	Test Concurrent Cash Addition and Removal	Verify that the cash counter correctly handles simultaneous operations.	The cash counter has a balance of 100.00.	1. Start two threads: one calling addCash(50.00) and the other calling removeCash(30.00). 2. Call checkBalance().	Returned balance = 120.00	(To be filled)	(To be filled)

EXPERIMENT 11

Objective: Develop test cases for various white box and black box testing techniques.

Testing Type	Test Case Description	Expected Output	Actual Output	Pass/Fail
White Box	Verify login function handles valid and invalid credentials using all code paths	Valid credentials log in successfully; invalid credentials show an error.	Valid credentials logged in; invalid credentials showed an error	Pass
Black Box	Verify login function with minimum, maximum, and empty input lengths.	Login succeeds with valid lengths; errors displayed otherwise	Errors for invalid lengths; successful login for valid lengths.	Pass
White Box	Ensure all branches of the ticket search function are executed.	Search results are displayed based on conditions.	Search results displayed as per conditions tested.	Pass
White Box	Verify data flow from cart to payment processing.	Payment is successfully processed, and tickets are confirmed.	Data flowed correctly, and payment succeeded.	Pass
Black Box	Test payment processing with all supported payment methods.	Successful payment for valid methods; error for invalid ones.	Payment succeeded for valid methods; error for invalid ones.	Pass
Black Box	Explore the behaviour of ticket search for misspelled city names and wrong date formats.	Correct suggestions or error messages are displayed.	Suggestions provided for misspelled cities; error for wrong date formats.	Pass

EXPERIMENT 12

Objective: Areas for Future Development

Future Enhancements:

The e-ticketing platform can be expanded to include additional features such as:

1. **Mobile App Integration:** Develop a mobile app for the e-ticketing platform to provide users with a seamless ticket purchasing experience.
2. **Multi-Factor Authentication with AI-Powered Biometrics:** Implement advanced security measures using AI-powered biometric authentication (e.g., facial recognition, fingerprint scanning) to ensure secure user authentication.
3. **International Transaction Support with Real-Time Currency Conversion:** Integrate real-time currency conversion using AI-powered APIs to facilitate seamless international transactions.
4. **AI-Driven Personalized Recommendations:** Implement a recommendation engine that uses machine learning algorithms to suggest relevant events, concerts, or shows to users based on their purchase history and preferences.
5. **Chatbots and Virtual Assistants for Enhanced Customer Support:** Integrate AI-powered chatbots and virtual assistants to provide 24/7 customer support, helping users with ticket-related queries and issues.
6. **Predictive Analytics for Demand Forecasting:** Utilize machine learning algorithms to analyze historical sales data and predict demand for upcoming events, enabling the platform to optimize ticket pricing and inventory management.

7. Computer Vision-Powered Ticket Validation: Implement a computer vision-based ticket validation system that uses AI-powered algorithms to verify the authenticity of tickets and prevent counterfeiting.

8. Natural Language Processing (NLP) for Sentiment Analysis: Use NLP techniques to analyze user feedback and sentiment, providing valuable insights for platform improvement and customer satisfaction enhancement.

These AI-powered features can enhance the e-ticketing platform's functionality, security, and user experience, making it more competitive and appealing to users.